Milestone 5: Mutation Testing and Test Suite Quality

CMPT 470/816: Advanced Software Engineering

University of Saskatchewan

Contents

1 Milestone Overview **2**

2 Learning Objectives **2**

3 Background **2**

4 Tool Description **3**
4.1 PIT Mutation Testing . . . 3

5 Required Dataset **3**
5.1 Assigned Repositories . . . 4
5.2 Analysis Scope: Selecting Target Classes . . . 5
5.3 Dataset Declaration Requirement . . . 7

6 Experimental Tasks **7**
6.1 Experiment 1: PIT Setup and Configuration (10 points) . . . 7
6.2 Experiment 2: Full Mutation Analysis (25 points) . . . 8
6.3 Experiment 3: Deep Surviving Mutant Analysis (30 points) . . . 8
6.4 Experiment 4: Mutation Operator Effectiveness (15 points) . . . 10
6.5 Experiment 5: Mutation Score Sensitivity (20 points) . . . 10

7 Report Requirements **11**
7.1 Introduction . . . 11
7.2 Background . . . 11
7.3 Experimental Setup . . . 11
7.4 Results . . . 11
7.5 Discussion . . . 12
7.6 Lessons Learned . . . 12
7.7 Conclusion . . . 12

8 Deliverables **12**

9 Grading Rubric **13**

10 Important Notes **13**

11 Important Notes **13**

12 Important Notes **14**

13 Tips for Success **14**

14 Resources **15**
14.1 PIT Documentation . . . 15
14.2 Recommended Reading . . . 15

1 Milestone Overview

Duration: Deadline is April 07, 2026
Weight: 11% of final grade
Software Engineering Area: Mutation testing, test quality assessment, behavioral reasoning about test suites.

This milestone asks you to go beyond running a tool and collecting numbers. The emphasis is on **reasoning**: you must explain *why* a mutant survived, *whether* it represents a real test weakness, and *how confident* you are in your conclusions.

You will use PIT to analyze your group's **assigned Java repository**, investigate surviving mutants in depth, and evaluate mutation operator effectiveness. It asks you to measure how sensitive mutation scores are to deliberate test suite changes—connecting mutation testing to the broader question of what it means for a test suite to be truly effective.

2 Learning Objectives

By completing this milestone, you will:

- Understand mutation testing principles and their role in test quality assessment
- Gain hands-on experience with PIT Mutation Testing
- Interpret mutation scores and construct evidence-based arguments about surviving mutants
- Distinguish equivalent mutants from genuine test weaknesses using code-level reasoning
- Evaluate test suite effectiveness beyond simple coverage metrics
- Analyze the sensitivity of mutation scores to targeted test suite modifications
- Evaluate cost-benefit tradeoffs of mutation testing in practice

3 Background

Code coverage (line coverage, branch coverage) tells us which code was executed during testing, but not whether tests can actually detect bugs. A test suite with 100% coverage might still miss critical defects if assertions are weak or missing.

Mutation testing assesses test suite quality by introducing small, artificial bugs (mutations) into the code and checking if the tests catch them:

1. **Generate Mutants:** Create modified versions of the code (like, change + to −, < to <=, remove method calls)
2. **Run Tests:** Execute the test suite against each mutant
3. **Classify Results:**
    - **Killed:** Test fails (detects the mutation)
    - **Survived:** All tests pass (mutation went undetected)

- **No Coverage:** Mutated code not executed by any test
- **Timeout/Error:** Infinite loop or crash induced

4. **Calculate Mutation Score:** $\frac{\text{Killed}}{\text{Killed+Survived}} \times 100$

A high mutation score indicates tests are effective at detecting introduced faults. A low score suggests weak assertions, missing test cases, or a high proportion of equivalent mutants.

Equivalent mutants are a fundamental challenge: a mutant that is semantically identical to the original program can never be killed by any correct test. Identifying them requires careful code-level reasoning, not just observation of PIT output.

4 Tool Description

4.1 PIT Mutation Testing

https://pitest.org/

PIT is a state-of-the-art mutation testing system for Java. It uses bytecode manipulation and coverage analysis to minimize test executions, making it practical for real projects. PIT integrates with Maven and Gradle and produces detailed HTML and XML reports showing which mutants survived and where.

Mutation Operators include:

- **Conditionals Boundary (COND_BOUNDARY):** change < to <=
- **Negate Conditionals (NEGATE_COND):** change == to `!=`
- **Math (MATH):** change + to -
- **Increments (INCREMENTS):** change `i++` to `i--`
- **Return Values (RETURN_VALS):** change return constants
- **Void Method Calls (VOID_METHOD_CALLS):** remove void method calls
- **Constructor Calls (CONSTRUCTOR_CALLS):** remove constructor calls
- Many more...

5 Required Dataset

Each group has been assigned the **same repository used in Milestone 4**. The repository is the **entire analysis scope**. You will run PIT on the full project (production code only) and report on all findings. All assigned repositories use either a Maven wrapper or a Gradle wrapper and have existing test suites.

You **must not** switch to a different repository. If you encounter build or PIT errors you cannot resolve, contact your TA with the full error log and your OS and Java version. A replacement repository will be provided if needed.

5.1 Assigned Repositories

- **Group: Turn it to 11**
  Assigned Repository: HikariCP
  URL: `https://github.com/brettwooldridge/HikariCP`

- **Group: Tech Titans**
  Assigned Repository: Gson
  URL: `https://github.com/google/gson`

- **Group: Group 6**
  Assigned Repository: jsoup
  URL: `https://github.com/jhy/jsoup`

- **Group: TryingTXL**
  Assigned Repository: Joda-Time
  URL: `https://github.com/JodaOrg/joda-time`

- **Group: The Coders**
  Assigned Repository: Apache Commons CSV
  URL: `https://github.com/apache/commons-csv`

- **Group: *(volatile team_t *)0 = "name"**
  Assigned Repository: Apache HttpComponents Client
  URL: `https://github.com/apache/httpcomponents-client`

- **Group: Hard Coded**
  Assigned Repository: Apache Commons Imaging
  URL: `https://github.com/apache/commons-imaging`

- **Group: Vibe Engineers**
  Assigned Repository: Jackson Databind
  URL: `https://github.com/FasterXML/jackson-databind`

- **Group: Silicon Valley's Finest**
  Assigned Repository: unirest-java
  URL: `https://github.com/Kong/unirest-java`

- **Group: Future Business Majors**
  Assigned Repository: Vavr
  URL: `https://github.com/vavr-io/vavr`

- **Group: Hopefully Graduating**
  Assigned Repository: Apache Commons Collections
  URL: `https://github.com/apache/commons-collections`

- **Group: No Name** (Members: Peng Wang, Saima Moonsat, Alan Rony, Zhiyang Teng, Meghal)
  Assigned Repository: Jackson Core
  URL: `https://github.com/FasterXML/jackson-core`

- **Group: Team Power Rangers**
  Assigned Repository: Apache Commons Lang
  URL: `https://github.com/apache/commons-lang`

5.2 Analysis Scope: Selecting Target Classes

You are **not required to run PIT on the full repository**. Instead, you must **select a justified subset** of the codebase. The quality of your justification is assessed as part of Experiment 1.

Why Selective Scope Matters

Running PIT on a large repository can generate thousands of mutants and take 30–90 minutes or longer. Selective scope lets you focus on the code where mutation testing yields the most insight—and makes the experiments feasible on a laptop in reasonable time.

Valid Selection Strategies

Below are some examples on how to justify your choice in the report:

1. **Feature-based selection**
   Select classes that implement a coherent feature or subsystem.
   Example: the parsing subsystem, the authentication layer, the serialization utilities.
2. **Layer-based selection**
   Select all classes in a single architectural layer.
   Example: the service layer, the utility layer, the data-access layer.
3. **Complexity-based selection**
   Select classes with high branching logic or high cyclomatic complexity, since these tend to produce the most interesting surviving mutants.
   Example: classes with multiple nested conditionals, state machines, or recursive algorithms.
4. **Test-coverage-gap selection**
   Select classes where you suspect tests are weak or missing—the mutation score will confirm or deny your hypothesis.
   Example: a class with high line coverage but few assertions, or a utility class that is exercised only indirectly.

What Counts as a Bad Selection

The following will result in deductions on Experiment 1:

- Selecting only trivial classes (pure getters, setters, data holders with no logic)
- Selecting a single very small class without a written justification
- Selecting code that has no associated unit tests
- Selecting scope so small that PIT finishes in under 30 seconds (see practical constraints below)

Required Justification Structure

Your report **must** include the following three items for the scope you selected. Vague statements such as "we chose this package because it seemed interesting" will not earn credit.

1. **What you selected**
   List the exact package(s) or class(es) used as `targetClasses` and `targetTests`.
2. **Why you selected it**
   Explain in terms of complexity, architectural importance, or testing concerns.
3. **Expected outcome before running PIT**
   State a hypothesis. For example: *"We expect a low mutation score because the existing tests use few assertions on return values."* or *"We expect a high kill rate because this package has dedicated unit tests for every public method."*

Reflect After Running PIT

After completing Experiment 2, you must revisit your hypothesis and answer in the report:

- Did the mutation score match your expectation?
- If the score was higher or lower than expected, why? What does this reveal about the test suite or the code under test?

This reflection is what turns a mechanical tool run into empirical reasoning. It is evaluated under the Experiment 2 discussion criterion.

Practical Constraints

Use the following as calibration for whether your scope is appropriately sized:

PIT runtime	Interpretation	Action
$<$ 30 seconds	Scope is too small	Widen to more classes or a larger package
30 seconds – 2 minutes	Marginal	Acceptable only with strong justification
2 – 30 minutes	Good	Ideal range for this assignment
$>$ 30 minutes	Possibly too large	Consider narrowing to a sub-package

Table 1: Scope calibration by PIT runtime

A practical starting point for most repositories: target **1–3 packages** or **3–5 classes**. If runtime exceeds 30 minutes, narrow to a single sub-package. If runtime is under 30 seconds, your scope is too small to produce meaningful results.

Configuration Flexibility

Because groups are selecting different scopes on different machines, you will need to tune your PIT configuration to match. The following parameters may need adjustment:

- `targetClasses` — the package or class pattern defining what is mutated
- `targetTests` — keep this broad (the full test package) so all tests can cover your selected scope
- `timeoutConstant` — raise to `8000` if you see many `TIMED_OUT` results; lower to `3000` if tests are fast
- `threads` — set to (CPU cores − 1); minimum 2
- `jvmArgs / -Xmx` — raise to `1024m` for larger scopes if you see `OutOfMemoryError` in the minion

Document all parameter values you used in Experiment 1. If you changed a value from the default, explain why.

5.3 Dataset Declaration Requirement

In your report, clearly state:

- Group name and assigned repository
- Repository URL and commit/tag used
- `targetClasses` and `targetTests` patterns
- Approximate LOC analyzed
- Total number of test cases in scope

6 Experimental Tasks

Complete **all** experiments.

6.1 Experiment 1: PIT Setup and Configuration (10 points)

1. **Install and Configure PIT:**
    - Add PIT plugin to Maven or Gradle build file
    - Configure `targetClasses` and `targetTests`
    - Enable all default mutation operators
    - Set reporting format to HTML + XML

2. **Pre-run Validation:**
   - Confirm all tests pass before any mutation run
   - Run PIT on a single class or small package to verify the setup
   - Attach a screenshot of the generated HTML report
3. **Document the following:**
   - PIT version and build tool version
   - Java version and OS
   - Full PIT configuration (you can attach as appendix if you want)
   - `targetClasses` and `targetTests` patterns used
   - Setup challenges encountered and how they were resolved

6.2 Experiment 2: Full Mutation Analysis (25 points)

1. **Run PIT on the full target scope:**

```
mvn org.pitest:pitest-maven:mutationCoverage
# or
gradle pitest
```

2. **Collect the following metrics from the PIT report:**
   - Total mutants generated
   - Mutants killed, survived, no-coverage, timed-out
   - Mutation Score (%): $\frac{\text{Killed}}{\text{Killed+Survived}} \times 100$
   - Test Strength (%): $\frac{\text{Killed}}{\text{Total Generated}} \times 100$
   - Line coverage (%)
   - Wall-clock execution time
3. **Mutation Operator Breakdown:** For each of the top 4 operators by mutant count, record:
   - Mutants generated and kill rate (%)
4. **Fill in the summary table:**

Project	LOC	Tests	Mutants Gen.	Killed	Survived	No Cov.	Mut. Score	Test Str.	Line Cov.	Runtime

Table 2: Mutation Analysis Summary

6.3 Experiment 3: Deep Surviving Mutant Analysis (30 points)

Select **three surviving mutants** for detailed investigation:

- **Mutant A:** one from the most frequent surviving mutation operator

- **Mutant B:** one from any different operator
- **Mutant C:** one that you suspect may be an equivalent mutant

For each mutant, record:

- Location (class, method, line number)
- Mutation operator applied
- Original code vs. mutated code (snippet)
- Screenshot or excerpt from the PIT HTML report

Mutant Reasoning Questions

Answer the following for **each** of the three mutants:

M1. Survival Explanation
Explain *precisely* why the existing test suite did not kill this mutant. Your answer must reference specific test methods (or the absence of them) and specific assertions. Saying "the tests did not cover this path" is insufficient unless you identify the missing coverage point and the test that would need to be extended or added.

M2. Semantic Impact Assessment
Describe a concrete runtime scenario in which the mutated version of the code would produce an observable difference in program output or behavior compared to the original. If *no* such scenario exists, argue why the mutant is equivalent. Your argument must cite the data flow and control flow context of the mutated line; general claims about operator semantics are not sufficient.

M3. Killability Verdict
Classify the mutant as one of:

- **Killable—Weak Assertion:** A test covers the mutated line but its assertion does not check the affected value
- **Killable—Missing Test:** No test exercises the execution path that reaches the mutation
- **Likely Equivalent:** The mutation does not change observable program behavior under any valid input; provide a formal or semi-formal argument (e.g., range analysis, constant folding, dead assignment)

Justify your verdict with code-level evidence.

Test Improvement (Required)

- For **at least two killable mutants** (Mutant A and one other), write or improve a test to kill the mutant.
- Re-run PIT and confirm both mutants are now killed.
- Report the change in mutation score (before and after).
- Explain why your new test is *sufficient* to kill the mutant, referencing the assertion you added.

Equivalent Mutant Evidence Table (fill in for Mutant C):

Criterion	Evidence
Mutated operator/statement	
Data flow context	
Why output is unchanged	
Counter-example attempted	
Verdict (equivalent / not equivalent)	

Table 3: Equivalent Mutant Analysis for Mutant C

6.4 Experiment 4: Mutation Operator Effectiveness (15 points)

Using the PIT HTML report:

1. **Rank the top 5 mutation operators** by kill rate (consider only operators with at least 10 generated mutants; kill rate = Killed / (Killed + Survived)).
2. **Identify 2 operators** with the lowest kill rates that reveal the clearest test weaknesses. For each:
   - Cite at least one specific surviving mutant from that operator
   - Explain what property of the test suite causes low kill rates for this operator class
3. **Discussion:**
   - Cost-benefit tradeoff: all operators vs. a selective subset
   - Under what project conditions is selective mutation testing preferable? Support your answer with data from your run.

6.5 Experiment 5: Mutation Score Sensitivity (20 points)

This experiment measures how responsive the mutation score is to **deliberate, targeted changes to the test suite**. This is analogous to asking: "If I improve my tests in a principled way, does the mutation score reflect that improvement faithfully?"

1. **Establish Baseline:** Record the mutation score from Experiment 2 as the baseline (S_0).
2. **Targeted Test Removal:** Remove or comment out **5 test methods** of your choice from the test suite. Choose tests you believe are directly responsible for killing mutants. Re-run PIT and record the new score (S_{remove}).
3. **Targeted Test Addition:** Restore the removed tests. Add **3 new test methods** designed to kill mutants that survived in the baseline run (these may be the tests from Experiment 3, but you must add at least one additional independent test). Re-run PIT and record the new score (S_{add}).
4. **Report and Analyze:**
   - Fill in the sensitivity table below
   - For each configuration, list which previously-alive mutants were killed or resurrected

5. **Answer the following questions:**

   S1. Did removing the 5 tests produce the score drop you expected? If the drop was larger or smaller than anticipated, explain why by examining which other mutants those tests were also killing.

   S2. Is the mutation score a *monotone* indicator of test suite quality in your experiment? That is, does adding more tests always increase the score? Provide evidence from your data.

   S3. What does this sensitivity analysis reveal about the *redundancy* or *uniqueness* of tests in your suite?

Configuration	Tests	Mutants Killed	Mutants Survived	Mutation Score (%)
Baseline (S_0)				
5 tests removed (S_{remove})				
3 tests added (S_{add})				

Table 4: Mutation Score Sensitivity

7 Report Requirements

Submit a **PDF report (6–8 pages, excluding appendices)**.

7.1 Introduction

Mutation testing overview, motivation, and milestone objectives.

7.2 Background

Mutation testing process, mutation operators, PIT's approach, equivalent mutants, and the difference from coverage-based metrics.

7.3 Experimental Setup

Environment details, project description, PIT configuration, test suite pre-validation, and setup challenges.

7.4 Results

Present results from all five experiments. Suggested figures:

- Bar chart: Mutation score and test strength
- Table: Mutation operator kill rates (Experiment 4)
- Sensitivity table (Experiment 5)
- Equivalent mutant evidence table (Experiment 3, Mutant C)

7.5 Discussion

Interpret your findings. Address: test suite quality, the reliability of the mutation score as an indicator, the practical cost of mutation testing at the scale of your chosen scope, and whether your pre-run hypothesis (Experiment 1) was confirmed or refuted by the results.

7.6 Lessons Learned

Key insights about test quality beyond coverage, challenges of equivalent mutant detection, and the implications of the sensitivity experiment.

7.7 Conclusion

Summary of findings and a concrete recommendation: given your results, under what conditions would you recommend mutation testing for a production Java project?

8 Deliverables

1. **PDF Report** — following the structure in Section 7
   File name: `GroupName_Milestone5_report.pdf`
2. **PIT Reports** — HTML reports for all three PIT runs (baseline, test removal, test addition)
3. **Improved and Added Test Source Files** — code for Experiments 3 and 5, clearly labeled per experiment
4. **Configuration Files** — PIT plugin configuration
5. **Contribution and AI Usage Statement**
   A plain-text (`.txt`) file containing:
   - Team name
   - Milestone name
   - Contribution of each group member (in percentages)
   - AI tool usage by each member, or an explicit statement that no AI tools were used
6. **Individual Contribution Report**
   Each group member must submit an individual contribution report *separately* to any TA by email. The report must include:
   - Your name and NSID
   - Team name
   - Milestone name
   - Your contribution (in percentages)
   - A brief comment on your team members' contributions

9 Grading Rubric

Component	Points	Criteria
Experiment 1: Setup	10	Environment fully documented; scope selection strategy stated; three-part justification complete (what, why, expected outcome); configuration documented; setup challenges resolved
Experiment 2: Analysis	25	PIT executed correctly, all metrics recorded accurately, summary table complete; pre-run hypothesis stated and post-run reflection addresses whether results matched expectations
Experiment 3: Mutant Analysis	30	Evidence-based reasoning for all three mutants; M1–M3 answered with code-level citations; equivalent mutant table completed; two mutants confirmed killed with score delta reported
Experiment 4: Operator Effectiveness	15	Accurate ranking; weak-assertion analysis backed by specific surviving mutants; cost-benefit discussion uses data
Experiment 5: Sensitivity	20	All three configurations run and reported; S1–S3 answered with evidence; monotonicity claim examined
Contribution and AI Statement	5	Complete, honest, submitted with group report
Individual Contribution Report	5	Submitted separately by each member
TOTAL	**110**	

Table 5: Milestone 5 Grading Rubric

10 Important Notes

1. AI tools may only be used for writing assistance.
2. All experimental results must come from actual tool execution.
3. Fabricated or manually constructed data will result in a zero grade.
4. Use the same repository for all five experiments.
5. Record exact tool versions for reproducibility.

11 Important Notes

1. **Execution Time:** Mutation testing is computationally expensive. Large projects may take hours. Start early and run overnight if needed.
2. **Test Requirements:** Projects MUST have passing test suites. Non-compiling tests will waste your time.
3. **Resource Usage:** PIT can be memory and CPU intensive. Close unnecessary applications. Configure thread count appropriately.
4. **Incremental Analysis:** PIT supports incremental mode to only test changed code. Use this for re-runs.

5. **AI Usage:** AI cannot run PIT for you. All analysis must be genuine. Submit PIT HTML reports as proof.
6. **Equivalent Mutants:** Identifying equivalent mutants requires code understanding. This is a known hard problem in mutation testing.
7. **Timeouts:** Some mutants cause infinite loops. PIT has timeout mechanisms. Document timeout counts.
8. **Version Control:** Use git to track test improvements.
9. **Selective Testing:** You can configure PIT to test specific packages/classes to reduce runtime.
10. **Report Files:** PIT generates HTML reports with detailed mutation information. Use these for analysis.

12 Important Notes

1. **Execution Time:** Mutation testing is expensive. Depending on the scope you select, runtimes can range from a few minutes to over an hour. Use the calibration table in Section 5.2 to size your scope appropriately. Use PIT's incremental mode (`withHistory=true`) for re-runs in Experiments 3 and 5.
2. **Green Suite Required:** All tests must pass before running PIT. Run `mvn clean test` and fix all failures before proceeding. PIT will refuse to run against a red test suite.
3. **Resource Usage:** PIT can be memory and CPU intensive. Close unnecessary applications. Set `threads` to (CPU cores − 1) and raise `-Xmx` if you see out-of-memory errors in the minion.
4. **Equivalent Mutants:** This is a known hard problem. A convincing argument based on data-flow reasoning earns full credit even if the conclusion is uncertain; an unconvincing one-line claim will not.
5. **Timeouts:** Document timeout counts. They factor into Test Strength but not Mutation Score.
6. **Fabricated Data:** Manually constructed or fabricated tool outputs will result in a grade of zero for the entire assignment.
7. **Same Repository and Scope:** Use the same repository and the same `targetClasses` pattern for all experiments. Changing scope mid-assignment makes results incomparable.
8. **Record Tool Versions:** Document exact versions of PIT, Java, Maven/Gradle, and the `pitest-junit5-plugin` for reproducibility.

13 Tips for Success

- **Read the PIT trace carefully.** For Experiment 3, understanding the mutation requires reading both the mutated line *and* the surrounding data flow—not just the operator name.
- **Use PIT's incremental mode** for re-runs after test changes to save time.

- **For the sensitivity experiment,** choose the 5 tests to remove strategically—remove tests with high kill counts to maximize the measurable drop.
- **For equivalent mutants,** trace the value of the mutated expression through all paths to any observable output or side effect. If the expression is dead, overwritten, or bounded, equivalence may hold.
- **Budget at least 2–3 hours for manual inspection** in Experiment 3. This is where most of the marks are.
- **Start Experiment 5 only after** Experiment 3 is complete, since you can reuse the tests you wrote.

14 Resources

14.1 PIT Documentation

- Official Site: `https://pitest.org/`
- Quick Start: `https://pitest.org/quickstart/`
- Maven Plugin: `https://pitest.org/quickstart/maven/`
- Gradle Plugin: `https://gradle-pitest-plugin.solidsoft.info/`
- Mutation Operators: `https://pitest.org/quickstart/mutators/`

14.2 Recommended Reading

- Coles, H. et al. (2016). PIT: a practical mutation testing tool for Java. *ISSTA 2016*, 449–452. `https://doi.org/10.1145/2931037.2948707`
- Zhu, Q., Zaidman, A., & Panichella, A. (2021). How to kill them all: An exploratory study on the impact of code observability on mutation testing. *Journal of Systems and Software*, 173, 110864. `https://www.sciencedirect.com/science/article/pii/S0164121220302545`
- Papadakis, M. et al. (2019). Mutation testing advances: An analysis and survey. *Advances in Computers*, 112, 275–378. `https://www.sciencedirect.com/science/chapter/bookseries/pii/S0065245818300305`